

План развития хранения и движения данных

От разрозненных файлов и ссылок — к понятным источникам истины, проверяемой целостности и быстрому интерфейсу.

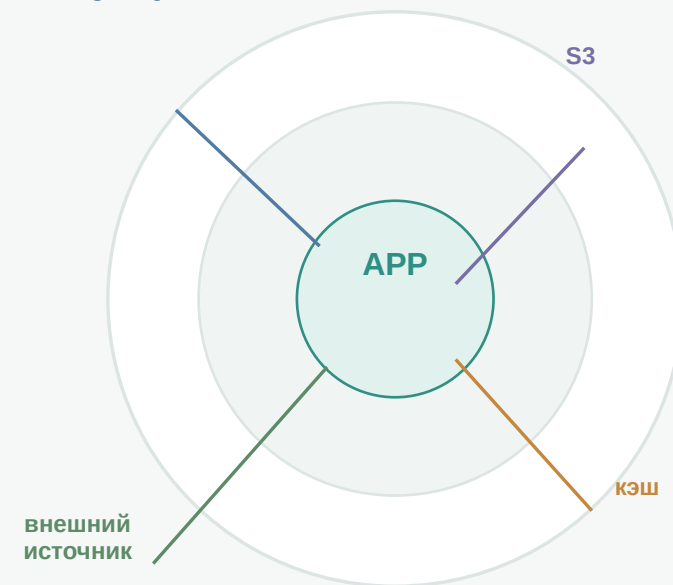
5 этапов

без big bang

с rollback

один source of truth

PostgreSQL



Главный принцип

**одно основное место
для каждого типа данных**

Почему систему нужно упорядочить

Сегодня приложение работает, но ответственность за данные распределена между слишком многими местами.

JSON + latest

актуальность
в двух местах

Внешние URL

файл может
исчезнуть

Локальный диск

каноника, кэш
и staging рядом

SQLite

локальный
управляющий контур

Что это создаёт

01

Решение может потерять связь
с замечанием после переаудита

02

Один файл имеет несколько
адресов и трактовок актуальности

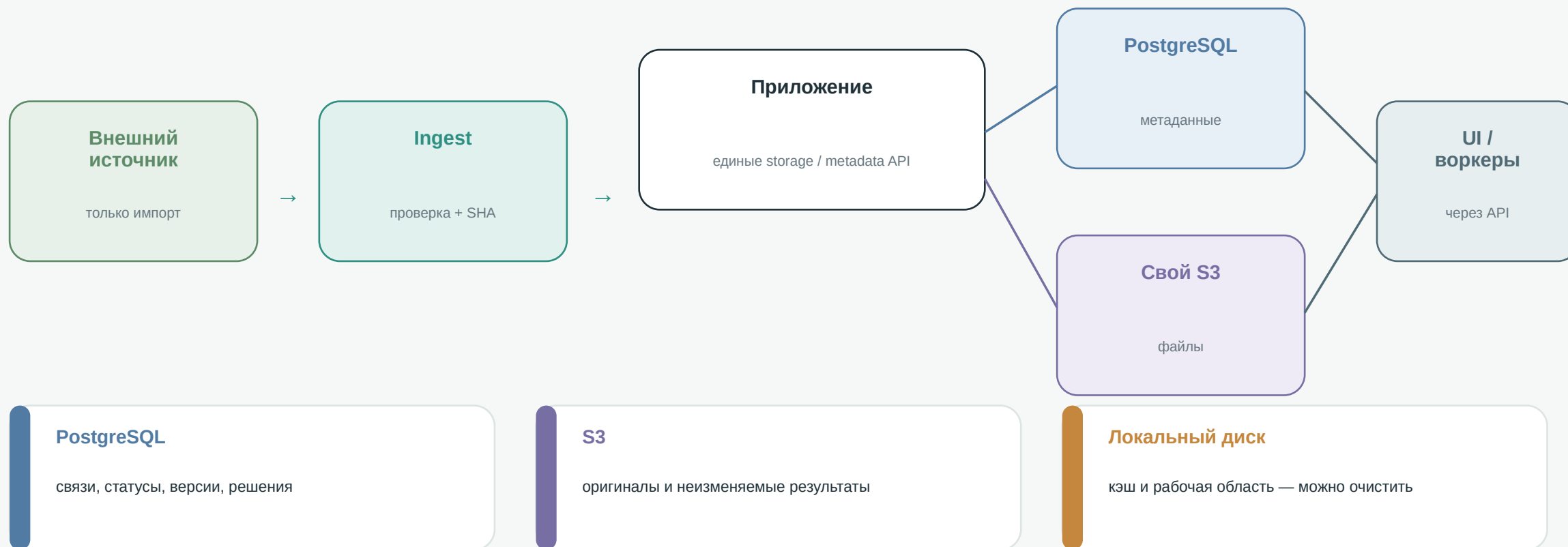
03

Сбой диска или внешней ссылки
становится потерей рабочего доступа

Приоритеты плана: целостность данных · меньше источников истины · предсказуемое быстродействие

Целевая модель: у каждого слоя одна роль

Метаданные, байты, импорт и временная обработка больше не конкурируют за статус каноники.



Производные данные и индексы всегда пересоздаются из этих источников.

Дорожная карта: сначала смысл, затем инфраструктура

Каждый этап уменьшает риск сам по себе и оставляет контролируемый возврат к предыдущему режиму.



2.1 · потоковый ingest

2.0–2.2 параллельно, без новой публикации

1-Б · projects_v2_primary

гейт production-записи этапа 2

Срочно до этапа 2

закрыть небезопасный export/download · проверить production auth и nginx

Нет массового удаления при миграции: shadow → parity → canary → primary → cleanup.

1. Единые правила идентичности

Не новая система вместо v2, а точная дельта к уже работающему projects_v2.

СМЫСЛ ЭТАПА

1

Было

Путь, latest и F-NNN
могут менять смысл



Стало

Стабильные UID и
неизменяемый run

Что меняем

- document_uid, version_id, run_id и finding_uid
- Manifest и один канонический current
- Самодостаточные решения эксперта и lineage
- 1-Б: запись переключается на projects_v2_primary

Что получаем

для ПОЛЬЗОВАТЕЛЯ

- Решение не теряется после переаудита
- История версии остаётся понятной

для СИСТЕМЫ

- Один смысл у каждой сущности
- Готовая основа для S3 и PostgreSQL

Дальше

После 1-Б можно безопасно включать production-запись файлового контура этапа 2.

2. Единый файловый контур, S3 и ingest

Один Storage Service управляет байтами; внешние источники остаются необязательными adapters.

СМЫСЛ ЭТАПА

2

Было

Пути, локальный диск
и временные URL



Стало

**blob_id + manifest +
private S3**

Что меняем

- 2.1: потоковый ingest без больших bytes в RAM
- 2.2: blob_id, manifests и Storage Service
- 2.3–2.4: S3 shadow, backfill и cutover
- 2.5: внешний S3/HTTPS только как adapters

Что получаем

для пользователя

- Документ не зависит от внешней ссылки
- Большие загрузки и PDF работают стабильнее

для системы

- Проверяемые immutable bytes
- Диск становится кэшем, adapter не блокирует план

Дальше

Стабильные blob_id и manifests становятся файловой основой PostgreSQL этапа 3.

3. PostgreSQL для метаданных

Связи и статусы переходят из файловых JSON в транзакционную модель с явными ограничениями.

СМЫСЛ ЭТАПА

3

Было

**JSON, указатели и
связи в каталогах**



Стало

**Таблицы, РК/FK и
транзакции**

Что меняем

- Таблицы объектов, документов, версий и gun
- Связи файлов по blob_id и логической роли
- Замечания, решения и история изменений
- Metadata Service, outbox, backfill и parity

Что получаем

для ПОЛЬЗОВАТЕЛЯ

- Одинаковая история во всех экранах
- Нет пропавших или дублированных связей

для СИСТЕМЫ

- Транзакции и ограничения целостности
- Параллельная работа и штатный backup

Дальше

Реальные SQL-профили дают основу для точной индексации этапа 4.

4. Индексация и ускорение чтения

Оптимизация следует за измерениями: ускоряются реальные пользовательские запросы.

СМЫСЛ ЭТАПА

4

Было

Тяжёлые запросы и
повторный рендер



Стало

SQL-индексы, views и
кэш производных

Что меняем

- Индексы под версии, статусы, даты и поиск
- Производные views для тяжёлых списков
- S3-кэш/CDN для миниатюр и страниц
- Профилирование P50/P95 и стоимости

Что получаем

для пользователя

- Быстрее открываются списки и документы
- Стабильнее работа на больших объёмах

для системы

- Нет постоянного обхода файлового дерева
- Ускорение подтверждено метриками

Дальше

После стабильного периода можно отключать legacy-read и временные projections.

5. Один источник истины и очистка наследия

Финальное переключение выполняется только после parity, restore drill и наблюдения.

СМЫСЛ ЭТАПА

5

Было

**Legacy-режим и
несколько зеркал**



Стало

**PostgreSQL + S3;
остальное — кэш**

Что меняем

- Чтение метаданных остаётся только в PostgreSQL
- S3 остаётся единственным источником файлов
- Legacy и _system выводятся по реестру
- Rollback-копии архивируются по retention

Что получаем

для пользователя

- Единое поведение во всех экранах
- Предсказуемая история и доступность

для системы

- Нет конкурирующих источников истины
- Кэши безопасно пересоздаются

Дальше

Целевая модель завершена: метаданные — PostgreSQL, байты — S3, диск — кэш.